drones

MDPI

Article

Adaptive Collision Avoidance for Multiple UAVs in Urban Environments

Jinpeng Zhang [1,2], Honghai Zhang [1,2,*], Jinlun Zhou [1,2], Mingzhuang Hua [2,3], Gang Zhong [1,2] and Hao Liu [2,4]

[1] College of Civil Aviation, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China; zjp0401@nuaa.edu.cn (J.Z.); jinlunzhou@nuaa.edu.cn (J.Z.); zg1991@nuaa.edu.cn (G.Z.)
[2] National Key Laboratory of Air Traffic Flow Management, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China; huamingzhuang@nuaa.edu.cn (M.H.); hliu@nuaa.edu.cn (H.L.)
[3] College of General Aviation and Flight, Nanjing University of Aeronautics and Astronautics, Liyang 213300, China
[4] College of Science, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China
* Correspondence: honghaizhang@nuaa.edu.cn; Tel.: +86-13914766151

Abstract: The increasing number of unmanned aerial vehicles (UAVs) in low-altitude airspace is seriously threatening the safety of the urban environment. This paper proposes an adaptive collision avoidance method for multiple UAVs (mUAVs), aiming to provide a safe guidance for UAVs at risk of collision. The proposed method is formulated as a two−layer resolution framework with the considerations of speed adjustment and rerouting strategies. The first layer is established as a deep reinforcement learning (DRL) model with a continuous state space and action space that adaptively selects the most suitable resolution strategy for UAV pairs. The second layer is developed as a collaborative mUAV collision avoidance model, which combines a three-dimensional conflict detection and conflict resolution pool to perform resolution. To train the DRL model, in this paper, a deep deterministic policy gradient (DDPG) algorithm is introduced and improved upon. The results demonstrate that the average time required to calculate a strategy is 0.096 s, the success rate reaches 95.03%, and the extra flight distance is 26.8 m, which meets the real-time requirements and provides a reliable reference for human intervention. The proposed method can adapt to various scenarios, e.g., different numbers and positions of UAVs, with interference from random factors. The improved DDPG algorithm can also significantly improve convergence speed and save training time.

Keywords: urban air traffic management; collision risk; adaptive collision avoidance; deep reinforcement learning; multiple unmanned aerial vehicles

Citation: Zhang, J.; Zhang, H.; Zhou, J.; Hua, M.; Zhong, G.; Liu, H. Adaptive Collision Avoidance for Multiple UAVs in Urban Environments. *Drones* **2023**, *7*, 491. https://doi.org/10.3390/drones7080491

Academic Editor: Pablo Rodríguez-Gonzálvez

Received: 5 July 2023
Revised: 20 July 2023
Accepted: 23 July 2023
Published: 27 July 2023

1. Introduction

Urban low-altitude airspace is an important natural resource that possesses great socio-economic value, and rational management of urban airspace is of great significance in alleviating traffic congestion and reducing the rate of ground traffic accidents [1,2]. As the main subjects of urban air traffic, unmanned aerial vehicles (UAVs) have attracted widespread attention due to their flexibility, convenience, and low cost. By the end of 2022, China had 700,000 registered UAV owners, 15,130 companies operating UAVs, 958,000 registered UAVs, and about 57,000 h of average daily flight [3]. It can be foreseen that with the development of urban air traffic, the types of tasks performed by UAVs will inevitably tend to diversify, showing great promise in fields such as tourism, rescue, and logistics, and with this comes an increase in urban air traffic flow and the complication of flight situations.

As the number and size of UAVs increases, their operation in urban airspace will present additional security threats. The dense distribution of buildings, the complex structure of the airspace, and the high density of aircraft make it extremely easy for accidents such as dangerous approaches or even collisions to occur. Therefore, in the face of limited airspace resources, means of effectively avoiding risk of collision have become a

primary issue that needs to be addressed in order to build urban air traffic demonstration areas and develop the low-altitude economy. However, the traditional collision avoidance algorithms lack sufficient success rates, and do not satisfy the safety interval criteria and real-time requirements in multi-target and high-density urban scenarios. In addition, these methods generate collision avoidance trajectories based on discrete state space, and the selectable actions are also discrete, and are not able to adequately reflect the flexibility of UAVs.

To address these problems, we propose an innovative two−layer resolution framework for mUAVs based on DRL, which can adaptively provide avoidance strategies for UAVs based on continue action space and ensure that each UAV has decision-making capability, thus significantly improving the success rate and computational efficiency.

1.1. Related Prior Work

Many studies have been performed proposing methods for UAV collision avoidance. In general, existing methods can be grouped into the following three categories: heuristic optimization methods, optimal control theory methods, and artificial intelligence methods.

(1) Heuristic optimization methods

Heuristic optimization methods divide the conflict process into a series of discrete state spaces and then perform an optimal search for approximate solutions in a certain cooperative manner [4], and primarily include the swarm intelligence optimization method [5] A*, D*. Zeng et al. combined the ant colony algorithms and the A* algorithm to solve the unmanned ground vehicle (UGV) scheduling planning problem, avoiding conflicts during simultaneous path planning for UGVs at a lower cost [6]. Zhao et al. considered collision probability and the intention information of intruders, using the A* algorithm to optimize trajectory planning to avoid collision risks [7]. Yun et al. applied the enhanced D* lite algorithm in the field of robot path navigation in unknown dynamic environments [8]. Furthermore, these methods are usually combined with other algorithms to solve the conflict problem, such as clustering methods [9] and Legendre Pseudo spectral Method [10].

(2) Optimal control theory methods

The optimal control theory methods select the permissible control rate according to the kinematic model or the time domain mathematical model so that the UAV operates according to the constraints and thus achieves collision avoidance. These methods mainly include three aspects of mixed-integer linear programming, nonlinear optimization, and dynamic programming. Radmanesh et al. proposed fast-dynamic Mixed Integer Linear Programming (MILP) for the path planning of UAVs in various flight formations, focusing on avoiding typical UAVs from colliding with any intruder aircraft [11]. De Waen et al. targeted complex scenarios with multiple obstacles, and divided the MILP problem into many smaller MILP subproblems for trajectory modeling, which ensures the scalability of MILP to solve conflict problems [12]. Alonso-Ayuso et al. developed an exact mixed-integer nonlinear optimization model based on geometric construction for tackling the aircraft conflict detection and resolution problem [13].

Heuristic-based search methods are reliable and effective for achieving collision avoidance and are able to resolve conflicts among small numbers of UAVs, which is the most common case in practice today. However, this method is not very suitable for mUAV conflicts, especially when the airspace is crowded, in which case the collision avoidance paths generated may suffer from secondary conflict problems. The optimal control methods take the minimum interval between UAVs as the optimization condition, and its relatively complex theory will lead to a decrease in anti-interference capability and an increase in computation, so it is not able to meet the real-time requirements.

(3) Artificial intelligence methods

The widespread use of artificial intelligence (AI) in recent years has provided new ideas and implementation paths. Reinforcement learning (RL) is the study of how an agent can interact with the environment to learn a policy that maximizes the expected

cumulative reward for a task. When RL is combined with the powerful understanding ability of deep learning, it is clear that it possesses better decision-making efficiency than humans in a nearly infinite state space [14,15]. The application of DPL to the field of UAV collision avoidance can solve the problems presented by the methods described above, while achieving better avoidance in urban airspace with variable environmental states and meeting strict real-time requirements.

The authors of [16,17] developed a Q-learning algorithm to design the dynamic movement of UAVs, but there is no assurance that it can handle high-dimensional input data. Singla et al. proposed a deep recurrent Q-network with temporal attention to realize the indoor autonomous flight of a UAV [18]. In [19], the DDQN algorithm was applied to ship navigation to achieve multi-ship collision avoidance in crowded waters. Li et al. designed a tactical conflict resolution method for air logistics transportation based on the D3QN algorithm, enabling UAVs to successfully avoid non-cooperative targets [20]. The value-based algorithms (e.g., D3QN and DDQN) can adapt to complex state spaces, but cannot provide satisfactory solutions for continuous control problems. The emergence of police-based RL has solved such problems; one of the most widely used and mature approaches in practice is the DDPG algorithm proposed by DeepMind [21]. For example, references [22–25] addressed the trajectory optimization in a two UAV scenario based on the DDPG algorithm. Ribeiro et al. [22] utilized a geometric approach to model conflict detection between two UAVs and trained the agent in conjunction with the DDPG algorithm to generate a resolution strategy. The authors of [23,24] utilized the DDPG algorithm to solve the UAV path following problem, taking into account the conflict risks during movement. In [25], a proper heading angle was obtained using the DDPG algorithm before the aircraft reached the boundary of the sector to avoid collisions. Alternatively, Proximal Policy Optimization (PPO) methods can be used in aircraft collision avoidance, and have shown a certain level of performance [26].

In summary, although a variety of UAV collision avoidance methods have been developed based on DRL, there are still several gaps in terms of actual application: (1) Scholars have typically rasterized the entire airspace when designing the state space [16,17,22], which has some specific limitations: UAVs can only move to an adjacent raster, which limits the action dimensions of UAVs, and is not able to adequately reflect their flexibility. The use of a discrete state space would cause a waste of airspace resources, and the whole raster area may become a no-fly zone due to some small buildings, thus reducing the space available for UAV flights. (2) The dimensional advantage is also an important factor in measuring the performance of the method, with most existing UAV collision avoidance methods borrowing from ground traffic, and thus the dimensional range is limited to 2D, which does not match the actual operation situation in the airspace, while also limiting the UAV avoidance actions that can be selected [18,20,26]. (3) When the DRL theory is applied to mUAV collision avoidance, in the existing literature, only one UAV is regarded as the agent, and the other UAVs are regarded as dynamic obstacles without resolution ability [20]; their tracks are previously planned. In actual operation, conflicts may arise from any UAV, so each UAV should have the ability to make their own decisions.

1.2. Our Contributions

To solve the above problems, in this paper, a more practical collision avoidance method for mUAVs is developed. The primary contributions of this study are the following:

(1) In this paper, an adaptive decision-making framework for mUAV collision avoidance is proposed. The adaptive framework enables UAVs to autonomously determine the avoidance action to be taken in 3D space, providing the UAVs with more options for extrication strategies when faced with static or dynamic obstacles. This framework combines the conflict resolution pool in order to transform mUAV conflicts into UAV pairs for avoidance, controlling the computational complexity at the polynomial level, thereby providing a new idea for mUAV collision avoidance.

(2) A DRL model for UAV pairs is designed based on a continuous action space and state space, reflecting the maneuverability of UAVs and avoiding wastage of urban airspace resources. The model endows each UAV with decision-making ability, and utilizes a fixed sector format to explore conflicting obstacles, thus simplifying the state of the agent and making it more adaptable to dense urban building environments.
(3) The DDPG algorithm is introduced to train the agent, and its convergence speed is enhanced by proposing the mechanism of destination area dynamic adjustment.

A summary of surveys related to DRL methods in the field of UAV collision avoidance is provided in Table 1. The table shows that this research represents the first study to resolve mUAV conflicts in a 3D environment based on continuous state and action space.

Table 1. Related work on conflict resolution based on DRL.

Ref.	Methods	Type	State and Action Space	Action	Multi-UAV	Environment
[16]	Q-leaning	Value-based	Discrete/Discrete	choosing from 4 possible directions	NO	2D
[17]	Q-leaning	Value-based	Discrete/Discrete	choosing from 7 possible directions	YES	3D
[18]	DDQN	Value-based	Continuous/Discrete	choosing from 3 possible directions	NO	2D
[20]	D3QN	Value-based	Continuous/Discrete	choosing acceleration; choosing yaw angular velocity	YES	2D
[22]	DDPG	police-based	Discrete/Continuous	choosing heading angle(max:15°/s); choosing acceleration (1.0 kts/s)	NO	2D
[23]	DDPG	police-based	Continuous/Continuous	choosing altitude, heading angle	NO	3D
[24]	DDPG	police-based	Continuous/Continuous	choosing heading angle $(-30°, 30°)$	NO	2D
[26]	PPO	police-based	Continuous/Continuous	choosing heading angle $(-30°, 30°)$,and speed [0 m/s, 40 m/s]	NO	2D
This paper	Improved DDPG	police-based	Continuous/Continuous	choosing velocity, heading angle, and altitude	YES	3D

The rest of the paper is organized as follows. In Section 2, the two−layer resolution framework is presented, and the methods for collision avoidance of UAV pairs and mUAVs are proposed. In Section 3, the improved DDPG algorithm is proposed for training the agent. In Section 4, the validity of the method is verified using a designed city scenario. In Section 5, some conclusions and summaries are presented. The table in the abbreviations section shows the main symbols used in this paper.

2. Problem Formulation

In this section, we propose a two−layer resolution framework (Figure 1) and divide the problem formulation of mUAV collision avoidance into two parts: collision avoidance between UAV pairs, and collaborative mUAV collision avoidance. Firstly, the collision avoidance agent training model is designed based on DPL, and the agent can assign avoidance actions for UAV pairs in real time and complete collision avoidance for both dynamic obstacles (between UAVs) and static obstacles (buildings). Secondly, the collaborative mUAV collision avoidance model is proposed, in which mUAV conflicts are transformed into the form of UAV pairs and then the agent is used to deconflict them one by one.

The collision zero model of the UAV and building are generated as described in an existing study [27], and are elliptical and cylindrical shapes, as shown in Equations (1) and (2).

$$D_u = \left\{ r \in R^3 : r^T A r \leq 1, A^{-1} = diag\left({a_u}^2, {b_u}^2, {h_u}^2 \right) \right\} \quad (1)$$

$$D_o = \left\{ (x, y, z) \in R^3 : x^2 + y^2 \leq {R_o}^2, z \leq h_o \right\} \quad (2)$$

where a_u, b_u, h_u are the semi-axes of the ellipsoid, respectively, and R_o, h_o are the radius and height of the cylinder.

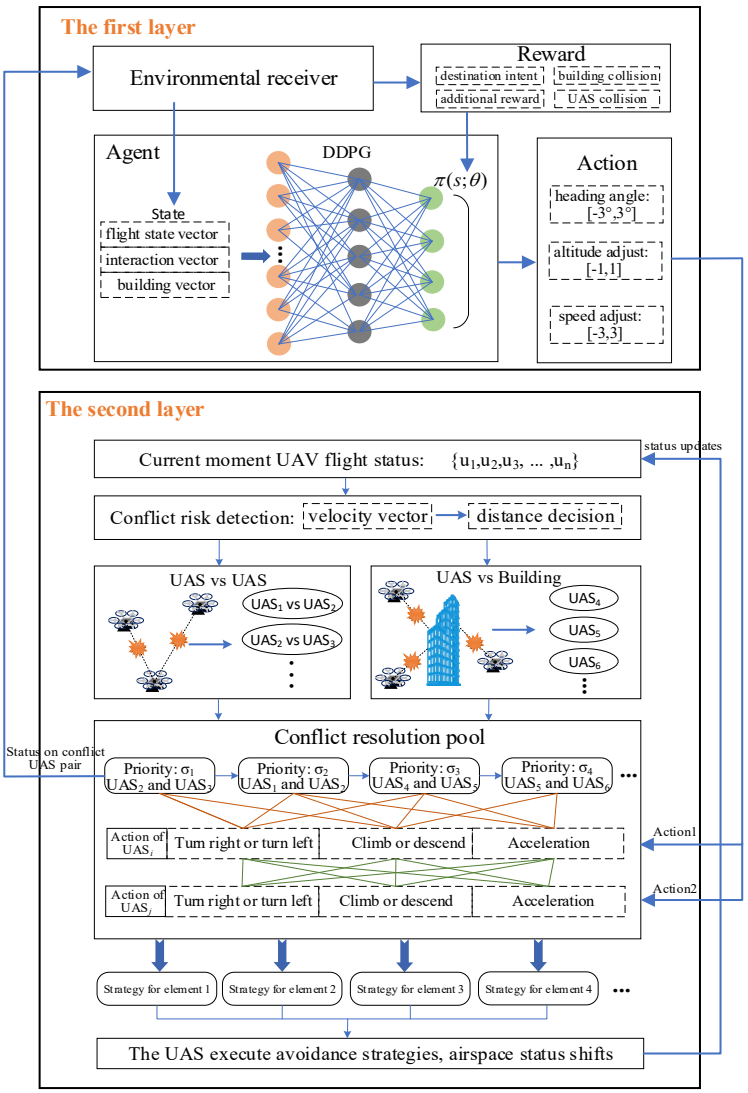


Figure 1. Overall workflow of the two−layer resolution framework.

2.1. First Layer: DRL-Based Method for Collision Avoidance between UAVs in a UAV Pair

In this section, the first layer of the framework is designed, and the agent training model is constructed with respect to three aspects: state space, action space, and reward function.

2.1.1. Continuous State Space

According to the explanation in the previous section, we replace the rasterized space with a continuous state space. The agent state vector includes the following three parts:

(1) The flight state vector of the UAV, including six attributes (φ: heading angle; V: horizontal speed; Z: the altitude of UAV; φ_g: relative heading angle of the destination to UAV; d_g: the horizontal distance of the destination to UAV), as shown in Figure 2. These attributes are able to accurately reflect the current flight status of the UAV, and the agent guides the UAV to its destination based on the flight status vector.

(2) Interaction vectors between UAV pairs (φ_{us}: the difference in heading angle; Z_{us}: the difference in altitude; d_{us}: the horizontal distance between two UAVs). These attributes

reflect the position and heading relationships between the UAVs in a UAV pair, and the agent avoids collision between UAVs in a UAV pair based on the interaction vectors.

(3) Building vectors. There are lots of buildings in the urban airspace, and if all building information is put into the agent, this will result in high state vector dimensionality and affect the speed of convergence. In this paper, considering the detection range of the UAV in the horizontal direction, a flight sector is used to map obstacles affecting flight into a fixed-length vector, and these are regarded as the obstacle vectors, as shown in Figure 2.

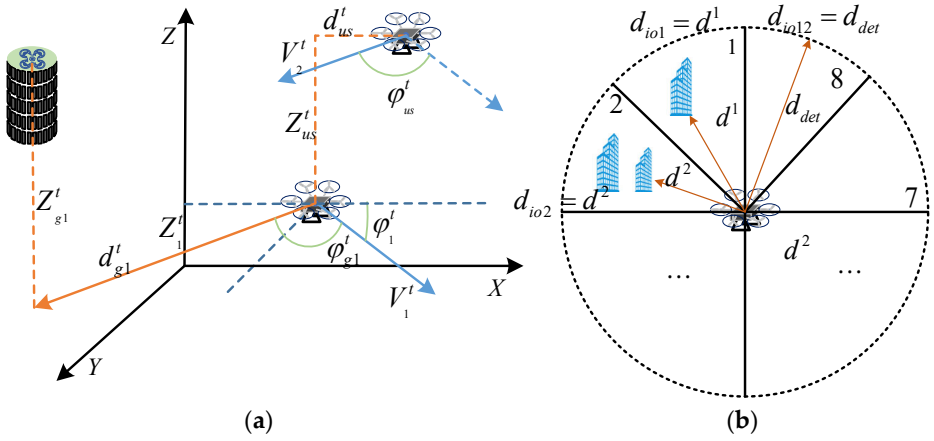


Figure 2. The diagram of the agent state vector. (**a**) Flight state vector. (**b**) Obstacle vectors.

The detection area is supposed to be a circle with the location as the center and the detection distance as the radius, that is, the agent can obtain the position information of obstacles in this area. Starting from the heading angle and rotating counterclockwise, every 45° is divided into one sector, and a total of eight sectors can be obtained. The distance between the UAV and all obstacles in each sector is calculated, and the closest distance is one of the attributes; if there are no obstacles in the sector, the value of this attribute corresponds to the detection distance. Supposing that there are j obstacles in sector m, then:

$$\begin{aligned} d^j_{iom} &= min\left(\left\|\left(x^j_{iom}, y^j_{iom}, z^j_{iom}\right) - \left(x^t_i, y^t_i, z^t_i\right)\right\| - R_u - R^j_o,\ d_{det}\right) \\ d_{iom} &= min\left(d^1_{iom}, d^2_{iom}, \cdots, d^j_{iom}\right) \end{aligned} \tag{3}$$

where R_u denotes the collision zone radius of the UAV, R^j_o denotes the collision zone radius of building j, d_{det} denotes the UAV detection distance, d^j_{iom} denotes the distance attribute between UAV and building j, d_{iom} denotes the distance attribute of sector m, so the obstacle vector of UAV is: $[d_{io1}, d_{io2}, \ldots, d_{io8}]$.

In summary, the state vector received by the agent from the environment at moment t is:

$$S_t = \left[\underbrace{\varphi^t_1, V^t_1, Z^t_1, \varphi^t_{g1}, Z^t_{g1}, d^t_{g1}}_{\text{State of UAS1}},\ \underbrace{\varphi^t_2, \ldots, d^t_{g2}}_{\text{State of UAS2}},\ \underbrace{\varphi^t_{us}, Z^t_{us}, d^t_{us}}_{\text{Interaction of UAS}},\ \underbrace{d^t_{1o1}, \ldots, d^t_{1o8}, d^t_{2o1}, \ldots, d^t_{2o8}}_{\textit{Obstacles} \text{ of UAS1 and UAS2}}\right], S_t \in S \tag{4}$$

2.1.2. Continuous Action Space

When modeling the UAV action space based on deep reinforcement learning, the control of the UAV is usually achieved based on adjusting spatial position, velocity, or acceleration. For the continuous state space and the flexibility of UAVs, in this paper, the continuous action space is designed based on adjusting the velocity and the maneuvering methods of UAVs are simplified, and are summarized into three processes: heading

adjustment, altitude adjustment, and speed adjustment ($\Delta\varphi$: alteration in heading angle; ΔZ: alteration in altitude; ΔV: alteration in horizontal speed), where at each time step, $\Delta\varphi \in [-3^\circ, 3^\circ]$, $\Delta Z \in [-1\,\mathrm{m}, 1\,\mathrm{m}]$,$\Delta V \in [-2\,\mathrm{m/s}, 2\,\mathrm{m/s}]$. Thus, the action space of the agent at moment t is:

$$A_t = \left[\underbrace{\Delta\varphi_1, \Delta Z_1, \Delta V_1}_{\text{Action of UAS1}}, \underbrace{\Delta\varphi_2, \Delta Z_2, \Delta V_2}_{\text{Action of UAS2}} \right], A_t \in A \tag{5}$$

2.1.3. Reward Function Design

The reward is a scalar feedback signal given by the environment that shows how well an agent performs at performing a certain strategy at a certain step. The reward function is a key component of the DRL framework. The purpose of the agent interacting with the environment is to maximize its reward value, so designing a suitable reward function for the agent can improve training performance and lead to faster convergence.

In this paper, we consider the shortest path to the destination, avoid collision between UAV pair, avoid collision between UAV and building, avoid the UAV flying out of the specific area, set four reward functions, and use artificially designed "dense" rewards to achieve a dynamic balance between near-term and long-term rewards, which can mitigate the sparse reward problem in DRL.

(1) Destination intent reward: when there are no obstacles in the sector of the UAV pair, the destination intent reward is used to ensure that the UAV takes the shortest path to the destination. The entire movement of the UAV is divided into multiple "intensive" actions, and the reward function is set for them to ensure that each action of the UAV affects the final reward value, thus contributing to the improvement of the overall strategy, as shown in Equation (6):

$$\begin{aligned} r_1^i &= \begin{cases} 0.5 & \varphi_{gi}^t \in [0^\circ, 10^\circ] \text{ or } \varphi_{gi}^t \in [350^\circ, 360^\circ] \\ 0.1 & \varphi_{gi}^t \in (10^\circ, 20^\circ] \text{ or } \varphi_{gi}^t \in [340^\circ, 350^\circ) \\ 0 & \varphi_{gi}^t \in (20^\circ, 90^\circ] \text{ or } \varphi_{gi}^t \in [270^\circ, 340^\circ) \\ -0.1 & \text{else} \end{cases} \\ r_2^i &= \left(d_{gi}^{t-1} - d_{gi}^t \right) + \left(Z_{gi}^{t-1} - Z_{gi}^t \right) \\ R_d^i &= r_1^i + r_2^i \end{aligned} \tag{6}$$

(2) Building collision avoidance reward: when there are obstacles in the sector of the UAV pair, it is necessary to ensure that the UAVs avoid colliding with buildings while flying to their destinations; therefore, we need to balance the two tasks, and the reward function at this time is shown in Equation (7).

$$\begin{aligned} r_1^i &= \begin{cases} 0.5 & \varphi_{gi}^t \in [0^\circ, 10^\circ] \text{ or } \varphi_{gi}^t \in [350^\circ, 360^\circ] \\ 0.1 & \varphi_{gi}^t \in (10^\circ, 20^\circ] \text{ or } \varphi_{gi}^t \in [340^\circ, 350^\circ) \\ 0 & \text{else} \end{cases} \\ r_2^i &= \left(d_{gi}^{t-1} - d_{gi}^t \right) + \left(Z_{gi}^{t-1} - Z_{gi}^t \right) \\ r_3^i &= \left(d_{iom}^t - d_{iom}^{t-1} \right) + \left(d_{iom}^t / d_{det} - 1 \right) \\ R_d^i &= r_1^i + r_2^i + r_3^i \end{aligned} \tag{7}$$

(3) UAV collision avoidance reward: the UAV collision avoidance reward is used to avoid collisions between the UAV and other UAVs. A UAV alert area is set up, and UAV

collision avoidance is made the main task when there are other UAVs within the alert area, as shown in Equation (8).

$$\begin{aligned} r_1^{ij} &= \begin{cases} -0.1 \cdot \left(\frac{\vec{V}_i^t \cdot \vec{V}_j^t}{\left|\vec{V}_i^t\right| \cdot \left|\vec{V}_j^t\right|} \right) - 1 & \underbrace{d_{us}^t < d_{det} \text{ and } Z_{us}^t < 4 \cdot H_u}_{Alerting\ Zone} \\ 0 & else \end{cases} \\ r_2^{ij} &= \begin{cases} \left(d_{us}^t - d_{us}^{t-1}\right) + \left(Z_{us}^t - Z_{us}^{t-1}\right) + \left(\frac{d_{us}^t}{d_{det}} + \frac{Z_{us}^t}{4 \cdot H_u} - 2\right) & Alerting \\ 0 & else \end{cases} \\ R_{uu}^{ij} &= r_1^{ij} + r_2^{ij} \end{aligned} \tag{8}$$

where $\vec{V}_i^t$ is the direction vector of the velocity of the UAV at moment t.

(4) Additional reward: There are four final states of the UAV: reaching the destination, flying out of the control area, colliding with obstacles, and colliding with other drones. This additional reward is used to provide a relatively large reward or penalty value when the UAV reaches its final state, which can be used to guide the UAV to its destination and avoid bad events such as collisions or loss of control, as shown in Equation (9).

$$R_{ex}^i = \begin{cases} 12 : U_{des}^i \in D_u^i \\ -6 : \text{ Drive out of control area} \\ -6 : D_o^j \cap D_u^i \neq \varnothing \\ -6 : D_u^j \cap D_u^i \neq \varnothing \end{cases} \tag{9}$$

In summary, the reward function received by the agent per unit of time is as follows:

$$R_t = R_d^i + R_{uu}^{ij} + R_{ex}^i \quad \forall i, j \in \{1, 2\} \tag{10}$$

2.1.4. The Interaction between the Agent and the Environment

In reinforcement learning, the interaction between the agent and its environment is often modeled by a Markovian decision process. This process can be represented by a four-tuple (S, A, R, γ), where S is the current state of the agent Equation (4), A is the action taken by the agent (Equation (5)), R is the reward value obtained by the agent after taking the current action, $\gamma \in [0, 1]$ is a discount factor, which is a constant, as shown in Figure 3.

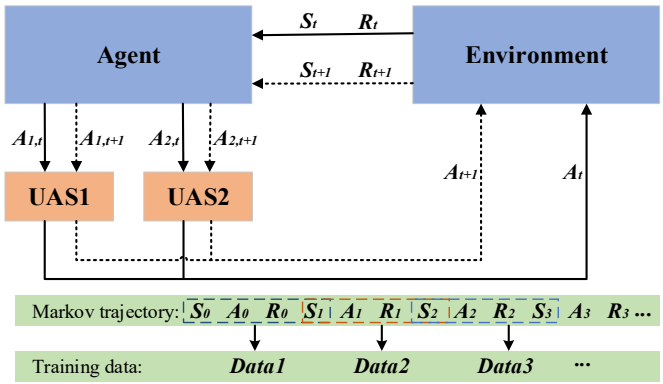


Figure 3. The interaction mode between the agent and the environment.

Assuming a discrete time domain $t \in \{0, 1, 2, \cdots\}$, the agent starts from the initial state S_0 and observes the environment state $S_t \in S$ at a certain time node t, taking action $A_t \in A$ based on the state and specific decision-making policy, and the next state is transferred to S_{t+1}, while the agent receives an immediate reward $R_t = R(S_t, A_t)$. All the variables obtained from the Markov decision process can be recorded as a trajectory

$\tau = (S_0, A_0, R_0, S_1, A_1, R_1, \cdots)$, and each segment of the trajectory can be intercepted to form a set of training data for the subsequent training of the Target-network and Evaluate-network in the algorithm.

2.2. Second Layer: Collaborative Collision Avoidance for mUAVs

2.2.1. Three-Dimensional Conflict Detection

To ensure mUAV safety, the conflict risk of each UAV needs to be detected, and then targeted for resolution. The conflict risk is detected based on the velocity vector and distance; the risk includes two types: UAV vs. UAV and UAV vs. building.

For conflict detection between the UAVs in a UAV pair, subscripts S and R designate the stochastic UAV and the reference UAV, respectively, in any UAV pair, P_R denotes the position of the reference UAV, P_S denotes the position of the stochastic UAV, and the velocities of the reference UAV and the stochastic UAV at the current moment are V_R, V_S.

In the process of modeling, defining a combined collision zone, which is assigned to the reference UAV so that the stochastic UAV can be regarded as a particle. The parameters of combined collision region D are:

$$H_D = 2H_u = 2\sqrt{3}h_u \quad R_D = 2R_u = 2\sqrt{3}a_u \tag{11}$$

A 3D collision coordinate system is established with the origin fixed at the position of the reference UAV; the relative position and velocity of the UAV pair are: $\Delta P_{uu} = P_S - P_R$, $\Delta V_{uu} = V_S - V_R$. It is assumed that Δl is the extension of the relative velocity ΔV_{uu}, and if the intersection of Δl and the combined collision region D is non-empty, i.e., $\Delta l \cap D \neq \emptyset$, then the UAV pair meets the condition of conflict in the relative velocity dimension; then, the spatial distance of the UAV pair is calculated, and if the spatial distance is less than the threshold while satisfying the relative velocity conflict condition, it can be determined that there is a collision risk for this UAV pair.

For conflict detection between UAVs and buildings, the combined collision zone is also defined. Since buildings are fixed, it is only necessary to consider whether the velocity extension line of the UAV intersects the combined collision region, and then the spatial distance between the UAV and building is considered to determine the conflict, as shown in Figure 4.

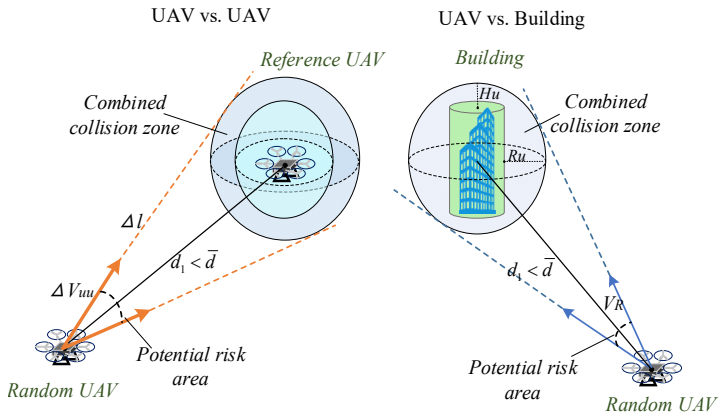


Figure 4. The diagram of three-dimensional conflict detection.

2.2.2. Conflict Resolution Pool

Based on the UAV conflict detection method in the previous section, we propose the concept of a conflict resolution pool, where the UAVs detected as being at risk are stored as the objects of the collision avoidance agent. For UAV-to-UAV conflicts, the elements in the pool are UAV pairs, and for UAV-to-building conflicts, the elements are single UAVs, while the distances of UAVs at risk are deposited in the pool as the priorities of conflict

resolution, with smaller distances indicating higher levels of risk and a more urgent need for collision avoidance, as shown in Equation (12).

$$P = \left\{ \underbrace{(u_1, u_2) : \sigma_{12}}_{Setelements}, \cdots, \underbrace{(u_i, u_j) : \sigma_{ij}}_{Conflict\ drone\ pairs}, \underbrace{u_3 : \sigma_3, \cdots, u_l : \sigma_l}_{\text{Conflict with buildings}} \right\} \sigma_{ij} = \frac{1}{d_{ij}}, \sigma_l = \frac{1}{d_l} \quad (12)$$

where (u_i, u_j) is the conflict UAV pair, u_l is the UAV in conflict with the building, σ is the priority, and d is the spatial distance.

The conflict resolution pool transforms mUAV conflicts into UAV pairs for avoidance, which simplifies the cooperative collision avoidance problem to a great extent. If a trajectory search-based approach is used to calculate the safe path for each UAV individually, the search space will grow exponentially with the number of UAVs, and when the number exceeds a certain value, the complexity of the algorithm will be too high to be able to solve the problem within an acceptable time. The conflict resolution pool prevents the resolution energy being wasted on temporarily safe UAVs, so that the computational complexity of the method can be controlled at the polynomial level, and the ability to perform collision avoidance will be further improved.

2.2.3. Collaborative Resolution Process for mUAVs

In this section, we propose a working model for this method by combining the concepts of the collision avoidance agent, three-dimensional conflict detection, and the conflict resolution pool.

Assuming that there are n UAVs in the urban airspace, the specific steps are as follows:

Step 1: A pool K is built, consisting of all UAVs in the airspace, which is initialized for each timestamp:

$$K = \{u_1, u_2, u_3, \cdots, u_n\} \quad (13)$$

Step 2: The three-dimensional conflict detection method is used to detect UAVs at risk, which are stored in the conflict resolution pool S, and then the priorities of the pool elements are calculated.

Step 3: The UAV pair with the highest priority is selected, as follows:

$$(i, j) = \underset{(i,j)}{argmax}(\sigma_{ij}) \quad (14)$$

If the UAV pairs are all in the pool K, the reinforcement learning agent is used to assign avoidance actions to them. For UAVs that are not in K, the actions assigned to it by the agent are ignored, and the original actions are kepts unchanged. Meanwhile, this UAV pair is removed from pool S.

Step 4: When there are no UAV pairs in the conflict resolution pool S, the two UAVs with the highest priorities that are in conflict with the building are selected and a UAV pair is formed, as follows:

$$(m, n) : m = \underset{l}{argmax}(\sigma_l), n = \underset{l'}{argmax}(\sigma_{l'}), m \neq n \quad (15)$$

The agent is used to assign avoidance actions to them, and the corresponding UAVs are removed from conflict resolution pool S.

Step 5: The UAVs that have been assigned avoidance actions are removed from pool K until $S = \varnothing$, and for the UAVs that are still in pool K, their original actions are kept unchanged.

Figure 5 shows the process of collaborative collision avoidance.

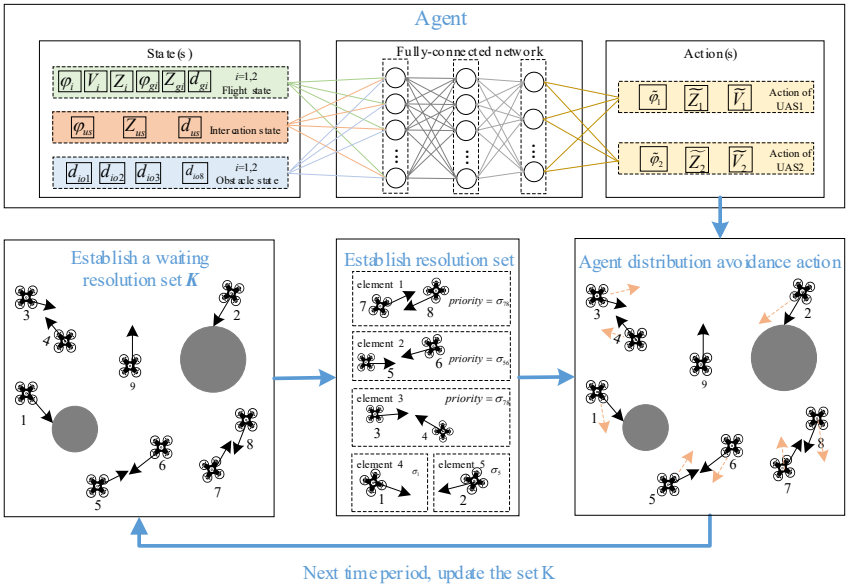


Figure 5. Diagram of collaborative collision resolution.

3. Improved Algorithm for Agent Training

3.1. Deep Deterministic Policy Gradient

Faced with a continuous state space and action space, in this paper, the DDPG algorithm is taken to train the collision avoidance agent for the UAV pair. The DDPG algorithm utilizes four neural networks in the actor-critic framework: policy network ($\pi(s;\theta)$), Q network ($Q(s,a;\omega)$), target policy network ($\pi(s;\theta^-)$), and target Q network ($Q(s,a;\omega^-)$).

The actor calculates the optimal action for the current state based on the learned policy function ($\pi_\theta(s_i)$). The critic estimates the value function ($Q_\omega(s,a)$) given the state and the action, which provides an expected accumulated future reward for this state–action pair. In addition, the critic is responsible for calculating the loss function (i.e., TD error) that is used in the learning process for both the policy network and the Q-network. To update the critic network, similar to Q-learning, the Bellman equation [28] is used:

$$target_t = R_t + \gamma Q(S_{t+1}, \pi(S_{t+1};\theta^-);\omega^-) \tag{16}$$

Then, the loss function is defined, and the argument is updated to minimize the *loss* between the original Q and the *target*:

$$Loss = \frac{1}{n}\sum_{t=1}^{n}(target_t - Q(S_t, a_t;\omega))^2 \tag{17}$$

The actor utilizes the policy network ($\pi(s;\theta)$) to select the best action, which maximizes the value function. The objective function in updating the actor is to maximize the expected return:

$$J(\theta) = \mathbb{E}(Q(s,a|\omega)|_{s=s_t,a=\pi_\theta(s_t)}) \tag{18}$$

According to the chain rule, the gradient of the objective function Q concerning the actor parameters can be obtained:

$$\nabla_\theta J(\theta) \approx \nabla_\theta \pi_\theta(s) \nabla_a Q(s,a) \tag{19}$$

Then, for mini-batch data, the mean of the sum of gradients is taken:

$$\nabla_\theta J(\theta) \approx \frac{1}{n}\sum_i \nabla_\theta \pi_\theta(s)|_{si} \nabla_a Q_\omega(s,a)\Big|_{s=si,a=\pi_\theta(si)} \tag{20}$$

The target network is a network used in the training phase. This network is equivalent to the original network being trained, and it provides the target values used to compute the loss function. In the DDPG algorithm, the target network is modified using a soft update:

$$\begin{cases} \theta^- \leftarrow \tau\theta + (1-\tau)\theta^- \\ \omega^- \leftarrow \tau\omega + (1-\tau)\omega^- \end{cases} \tag{21}$$

This means that the target weights are constrained to change slowly. The use of target networks with soft updates allows them to give consistent targets during the temporal-difference backups and causes the learning process to remain stable.

3.2. An Improved Measure for DDPG

The DDPG algorithm has high execution efficiency, enabling continuous motion control of the agent. However, presented with the specific environment described in this paper, the DDPG consumes too much time in agent training, making it difficult to respond quickly when the urban environment undergoes significant changes and the agent needs to be retrained. To address such problems, this section improves the algorithm mainly in terms of the dynamic adjustment of the destination area.

Due to the large spatial area of the city, the UAV destination is relatively small, and is replaced by a prime point in Equation (9), where $U^i_{des} \in D^i_u$ indicates that the UAV has reached its destination, and the agent receives the corresponding reward. However, in the actual training process, it is difficult to achieve the above conditions when the UAV performs the search, so there is little chance for the agent to obtain a relatively large reward value, thus causing the convergence speed to decrease.

In this section, the way of obtaining destination rewards in the algorithm is improved based on the Wright learning curve model, and a dynamic adjustment mechanism for the destination area is proposed. During the early stage of training, the destination area is expanded so that the agent can complete the task relatively easily and learn the primary strategy. According to the learning curve, the destination area is gradually reduced, and the agent gradually learns more difficult strategies, which is conducive to improving the stability of the learning and accelerating the convergence speed of the algorithm. The destination range is defined as being spherical:

$$U^i_{des} \in D^i_{des}, D^i_{des} = \left\{ r \in R^3 : r^T r \le R^i_{des} \right\} \tag{22}$$

In Equation (22), D^i_{des} represents the destination area, if $D^i_u \cap D^i_{des} \neq \varnothing$ means the UAV has reached its destination, the area radius is adjusted with the training episode according to the Wright learning curve model:

$$\begin{cases} \alpha = \frac{\lg C}{\lg 2} \\ R^i_{des} = \overline{R} \cdot x^{\alpha} \end{cases} \tag{23}$$

In Equation (23), α is the learning rate, C is the attenuation coefficient, $\overline{R}$ is the initial destination area radius, and x indicates the training episode. The dynamic adjustment mechanism of the destination area further optimizes the "dense" reward and allows the algorithm to learn useful experiences in the early stages.

The original DDPG algorithm needs more training epochs to detect the accurate location due to the small and fixed destination area, and sometimes even fails to obtain the destination reward. The improved DDPG algorithm adds a dynamic adjustment mechanism for the destination area, which enlarges the size of the destination area in the initial stage of training, so that the agent can easily obtain the approximate destination location, ensuring that it will move in the right direction in the subsequent training. As the training progresses, the algorithm gradually reduces the destination area based on the Wright learning curve, guiding the agent to the precise destination location. Compared

with the original algorithm, the improved DDPG algorithm is more goal oriented and avoids ineffective exploration on the part of the agent, so it can accelerate the convergence speed and save training resources.

4. Results and Discussion

4.1. Environment Setting and Hyperparameters

To analyze the performance of the mUAV collision avoidance method, in this paper, a DJI Matrice 600 is selected as a case study, whose form factor (L × W × H) is set to 1668 mm × 1668 mm × 759 mm, and the calculated elliptical collision zero parameters are 1445 mm × 1445 mm × 657 mm. The scenario range is set to 1000 m × 1000 m × 50 m, considering that the collision avoidance area of small UAVs in the city will not be too large.

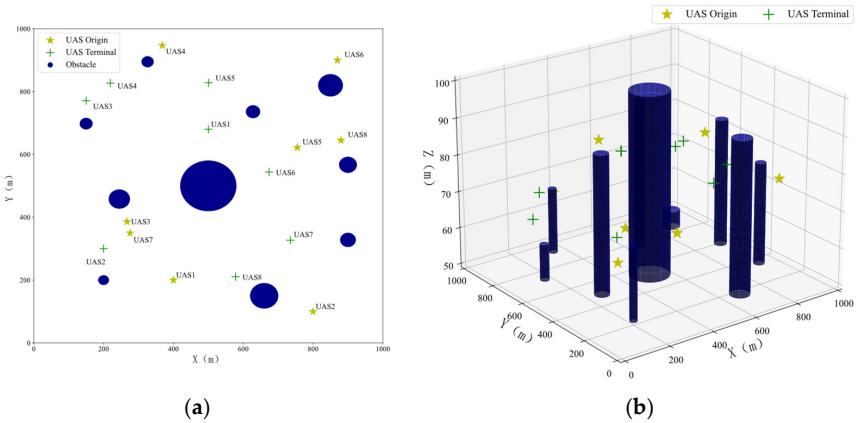
In the experimental scenario, we construct the spatial layout of buildings in the city and set up fixed-volume obstacles at fixed locations; the building collision zero is cylindrical, and the shape parameters are shown in Table 2. The experiment was based on eight UAVs, each with an initial speed and initial heading. At the beginning of training, a random origin is generated for each UAV and the terminal, and the agent assigns avoidance actions to UAVs according to the action space in Equation (5), and returns to the origin to restart the training if a UAV has a collision accident. The experimental parameters are as shown in Table 3, and the environment is as shown in Figure 6.

Table 2. The shape parameters of the building.

Obstacle Number	X (m)	Y (m)	*R* (m)	Z (m)	Obstacle Number	X (m)	Y (m)	*R* (m)	Z (m)
1	500	500	80	100	6	245	458	30	89
2	200	200	15	70	7	660	150	40	56
3	900	567	25	85	8	900	328	22	78
4	850	820	35	80	9	326	895	17	78
5	150	698	18	60	10	628	736	20	91

Table 3. Parameter value.

Parameter	Value
Total number of training episodes	5000
Discount factor	0.99
Target network update rate	0.001
Buffer size	10,000
Batch size	100
The initial destination area radius: $\overline{R}$	10
Attenuation coefficient: *C*	0.8



(**a**) (**b**)

Figure 6. The experimental environment. (**a**) Two-dimensional perspective. (**b**) Three-dimensional perspective.

4.2. Collision Avoidance Agent Training

For various conflicts arising from UAV pairs, the trained agent can provide appropriate solutions. During training, the average reward obtained per episode is an important indicator of convergence and collision avoidance performance. Using the improved DDPG algorithm to train agents, the rewards for each episode are shown in Figure 7.

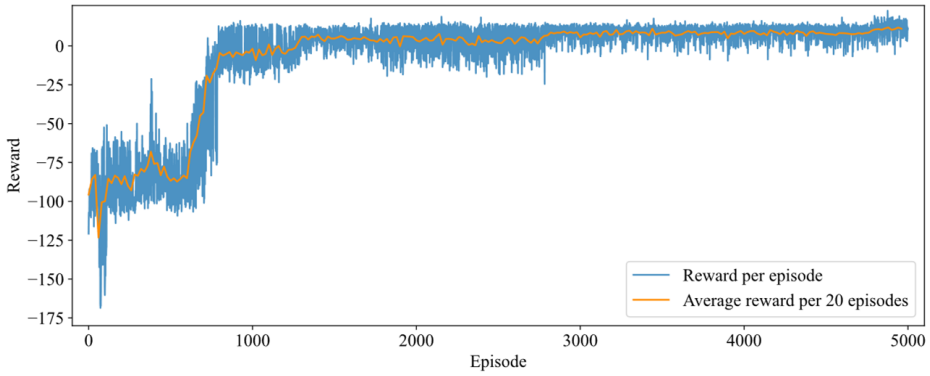


Figure 7. The rewards for each episode during training.

From Figure 7, the reward obtained by the agent is not stable at the beginning of the training, as the agent touches the events with a higher degree of punishment during the exploration process, resulting in a large degree of reward drop. With continuous training, the agent gradually learns the high-reward behavior, and the reward value increases. In the second half of training, the reward did not significantly fall again, which indicates that the improved algorithm learned a better and more stable strategy, and therefore the reward oscillated less.

The comparison effect of the improved DDPG algorithm with the original algorithm is shown in Figure 8. It can be seen that the improved algorithm demonstrates a better improvement in convergence speed, achieving a higher reward value and showing a convergence trend of around 380 training epochs, while the original algorithm was only able to show such an effect after around 1000 epochs. After the 2000th training epoch, the rewards obtained by the two algorithms did not differ much. However, as for actual training, the improved DDPG algorithm obtained stable reward values and determined the convergence trend at earlier epochs, and thus training can be ended earlier than in the original algorithm, which saves training time.

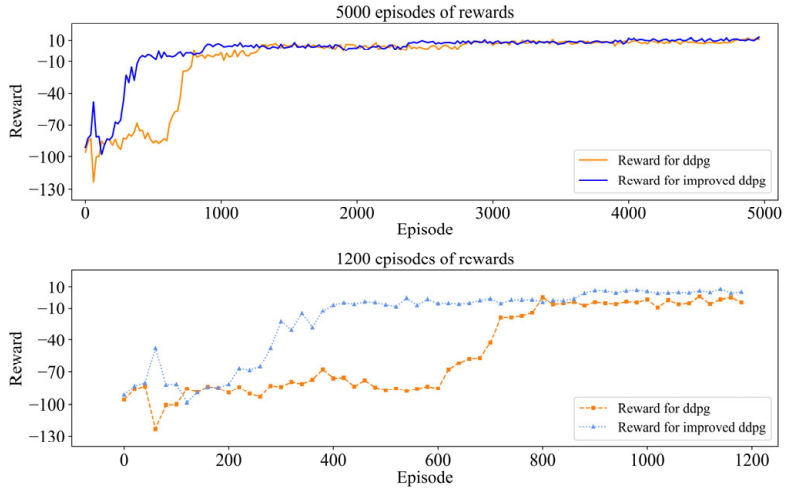


Figure 8. Comparison of the convergence process.

4.3. Numerical Results Analysis

4.3.1. Collision Avoidance Results

Using the two−layer resolution framework, we obtained the collision avoidance results, as shown in Figure 9, while recording the distance between each UAV and the nearest obstacle, as well as the distance between the two nearest UAVs.

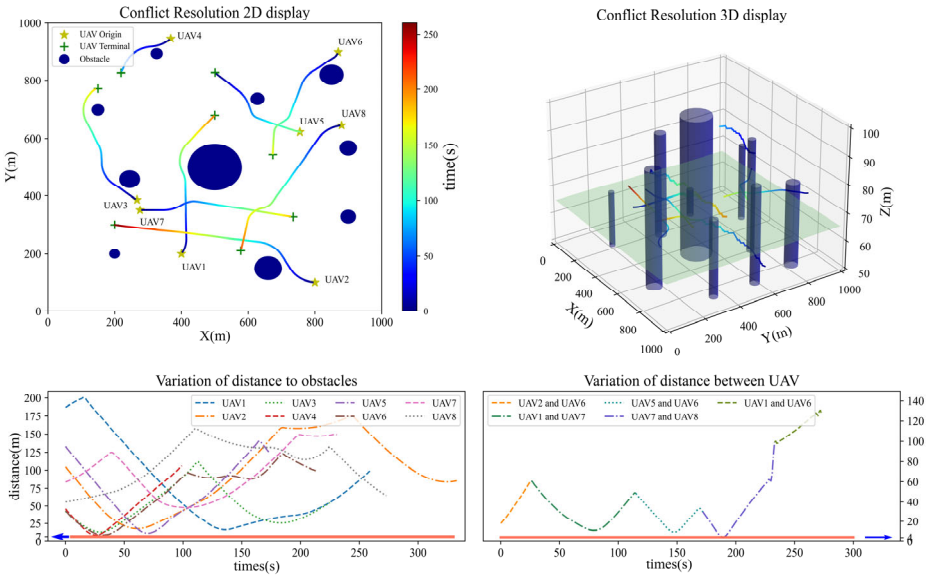


Figure 9. Collision avoidance results.

From a two-dimensional perspective, it is intuitively apparent that every UAV is in conflict with at least one obstacle and avoids obstacles with as little extra flight distance as possible. In addition, there is a risk of conflict between UAV5 and UAV6, UAV7 and UAV8, and UAV1 and UAV7, so the agent randomly selects one of the UAVs to perform the primary avoidance maneuver, while the other maintains almost its original direction (or makes minor adjustments), in order to minimize the impact of the avoidance behavior on normal navigation. In the three-dimensional view, all the UAVs have reached the intended altitude.

The distance between each UAV and the obstacle has a process from small to large, which indicates that UAVs are making avoidance actions. The closest UAV pairs may be different at different times, but the overall trend of distance variation is consistent, proving that the UAVs can also avoid each other. The minimum distance from buildings is about 7 m, and the minimum distance from other UAVs is about 4 m throughout the whole process, thus meeting the standard safety interval, proving that the model in this paper can ensure the safe operation of UAVs in cities with many buildings.

4.3.2. Avoidance Strategy Analysis

In the two−layer resolution framework, three strategies are used for collision avoidance and destination guidance, to analyze the avoidance action selection pattern by the agent, recording the actions (heading angle, altitude, speed change) selected by all UAVs at each step, as shown in Figures 10–12.

As shown in Figure 10, each approach of UAVs and obstacles will lead to a significant change in the heading angle, and when the distance is kept at a relatively safe level, the change in heading angle will fluctuate around 0°, indicating that the UAV is flying along a straight line in the horizontal direction. It can be determined that the agent avoids collision with obstacles mainly by changing the heading angle of the UAV.

As shown in Figures 11 and 12, climbing and descending actions ensure that the height of the UAV is finally consistent with the destination height, demonstrating that the agent has the guiding ability in the three-dimensional space. The speed change is generally stable

within a fixed range, and there is no excessive speed, as the speed adjustment is coupled with the heading angle to avoid collision with obstacles. In addition, due to there being fewer obstacles near the destination, the UAV has a higher speed and a stable heading angle in the later stage, ultimately reaching the destination.

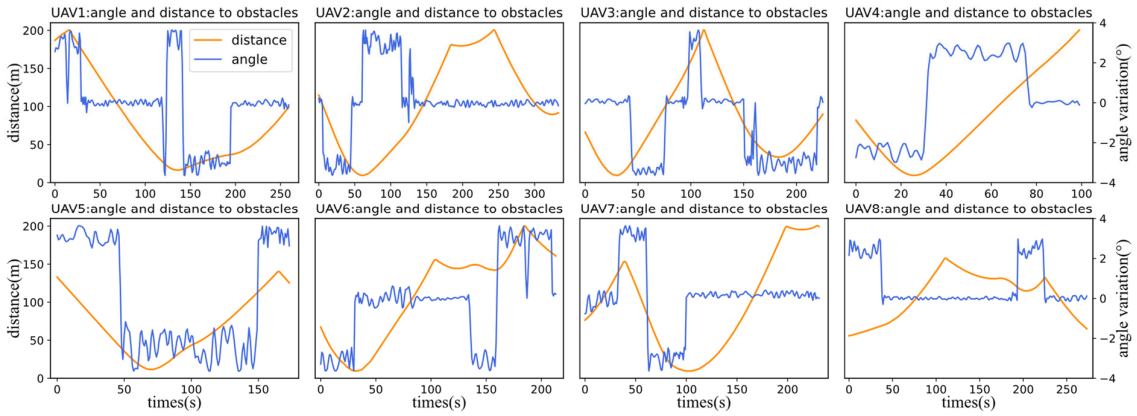


Figure 10. The heading angle change trend and distance to obstacles of UAV.

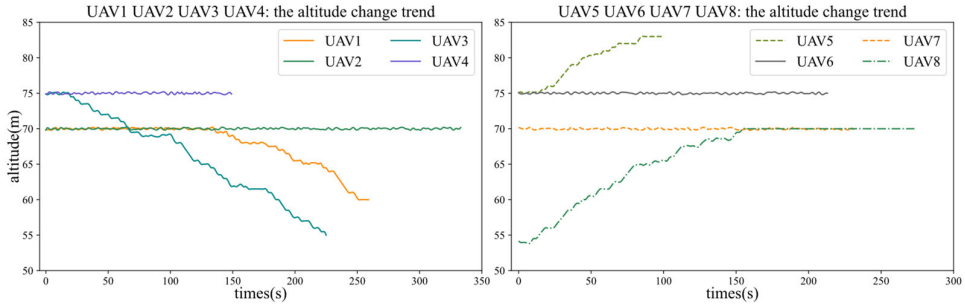


Figure 11. The altitude change trend of UAV.

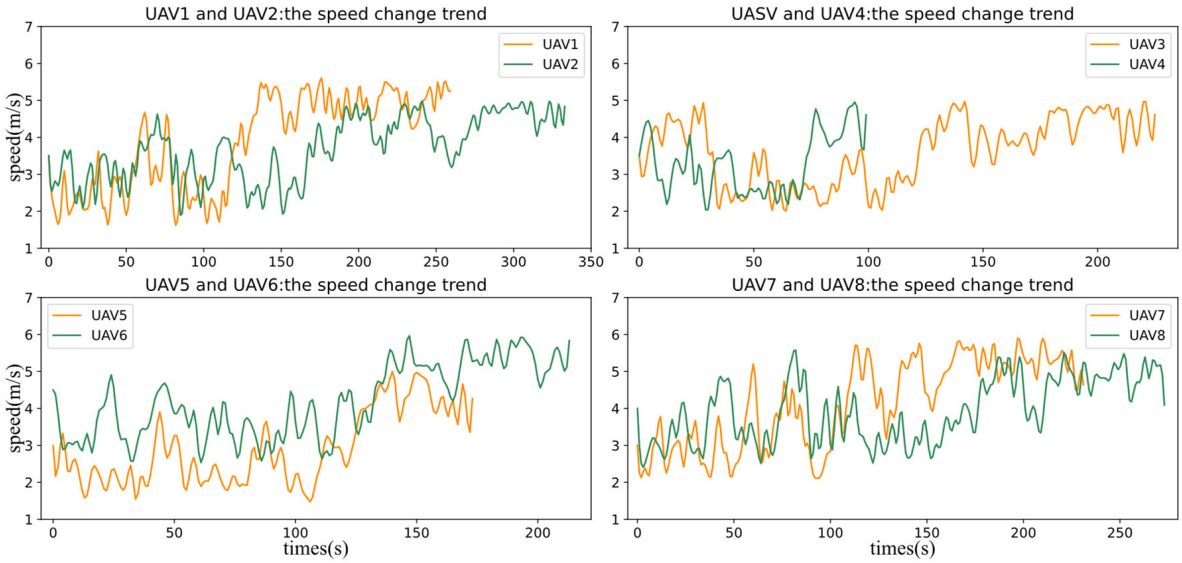


Figure 12. The speed change trend of UAVs.

4.4. Performance Analysis

4.4.1. Performance Testing of the Method

To fully illustrate the collision avoidance effect of the method, 300 different scenarios were designed in the simulation area by randomly generating the starting and ending

points of eight UAVs. The considered performance metrics were the following: (1) collision avoidance success rate (SR), which is the percentage of successful collision avoidance; (2) computational efficiency (CE), which is the time required for an agent to calculate an action; (3) extra flight distance (ED), which is the record of extra distance the UAV flighted due to collision avoidance.

(1) Collision avoidance success rate

In the 300 random scenarios, 4523 conflicts were recorded in the conflict resolution pool, including 867 conflicts with other UAVs and 3656 conflicts with buildings. If the agent cannot assign the correct avoidance action to a UAV, a collision will occur, according to the current speed trend. The collision avoidance method described in this paper can guide UAVs out of collision risk, with success rates as shown in Table 4.

Table 4. Success rates of collision avoidance.

	With Buildings	With Other UAVs	Total
Number of conflicts	3656	867	4523
Number of resolutions	3502	796	4298
Success rate	95.79%	91.81%	95.03%

The data in Table 4 show that the method has a higher success rate when resolving conflicts with fixed obstacles than with dynamic obstacles, as the invading UAV has positional uncertainty, the flight state may not be fully perceived by the current UAV, and no avoidance action can be taken in time. However, the overall success rate reached 95.03%, indicating that the method is able to guide UAVs to avoid most collision risks and can provide an adequate and reliable reference for urban air traffic management.

(2) Computational efficiency

Recording the total computation time and the number of avoidance actions performed by UAVs in each scenario, to calculate the average time that the method to plan an action for a UAV, and this is used as the evaluation index for computation efficiency, as shown in Equation (24).

$$T_f^j = \frac{T_{total}^j}{\sum_{i=1}^{8} num_i^j} \tag{24}$$

where T_{total}^j is the total time required to calculate an avoidance action in scenario j, and num_i^j is the number of avoidance actions performed by UAV i.

The average time required for each scenario is shown in Figure 13. From the figure, the avoidance action calculation time of 300 scenarios is at the 0.01 s level, with an average time of 0.0963 s, which can meet the real-time requirements of collision avoidance.

(3) Extra flight distance

When facing obstacles, the agent guides the UAV to change its heading or speed, which adds extra flight distance (ED) compared with the original trajectory. The metrics of ED are used to measure the impact of collision avoidance on UAVs, as shown in Equation (25):

$$d_e = \frac{1}{n}\sum_{i=1}^{n}(d_{oi} - d_{ni}) \tag{25}$$

In Equation (25), d_e means the extra flight distance (ED), d_{oi} means the distance of the i-th trajectory directly to the destination regardless of any conflicts, d_{ni} means the distance of the i-th trajectory which has considered the collision avoidance. n is the total number of trajectories in all scenarios.

From the perspective of flight efficiency and green transportation, the shorter the ED, the less flight energy is lost, and the less impact there is on the original flight [29]. The average extra flight distance of eight aircraft in 300 scenarios is 26.8 m, which is relatively good and is acceptable in terms of a collaborative resolution process for mUAVs.

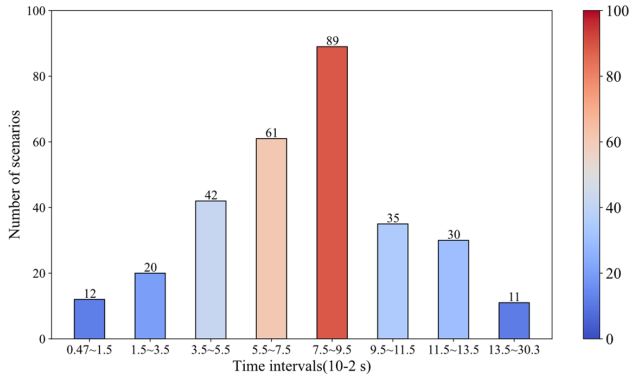


Figure 13. The average time required for allocating actions in each scenario.

4.5. Impact of Noisy States

UAVs may have positional uncertainty due to the interference of random factors such as crosswinds, which can affect collision avoidance behavior. To investigate the robustness of the proposed method, we performed simulations with noisy states, adding noise to the state information of UAV. It was assumed that each noise component was uniformly distributed, i.e., $m = [m_x, m_y, m_z, m_v]$, $m \sim U(-\varepsilon, \varepsilon)$. The noise was added to the state information of UAV, i.e.,$P = [x_i^t + m_x, y_i^t + m_y, z_i^t + m_z]$ and $\overline{V}_i^t = V_i^t + m_v$, which will change the state of the agent in Equation (4).

Table 5 shows the SR and ED performances with noisy states. The table shows that noise influences the SR and ED performance, meaning that the UAV does not have accurate position and velocity information, leading to biased observations by the agent, which may output incorrect avoidance actions. However, even if noise $\varepsilon = 3$ is added to both position and velocity information, SR is, at most, 91.61%, and ED is 36.2 m, and thus a good level is still maintained. Therefore, our method has high tolerance to noisy observations.

Table 5. SR, and ED performance when states are noisy.

	$mx, my, mz \neq 0$ $vx = 0, \varepsilon = 0.5$	$mx, my, mz \neq 0$ $vx = 0, \varepsilon = 1.5$	$mx, my, mz \neq 0$ $vx = 0, \varepsilon = 3$	**No Noise**
SR	94.32%	93.98%	92.90%	95.03%
ED	27.2 m	30.3 m	35.6 m	26.8 m
	$mx, my, mz = 0$ $vx \neq 0, \varepsilon = 0.5$	$mx, my, mz = 0$ $vx \neq 0, \varepsilon = 1.5$	$mx, my, mz = 0$ $vx \neq 0, \varepsilon = 3$	
SR	94.54%	93.52%	93.3%	
ED	27.5 m	29.6 m	34.3 m	
	$mx, my, mz \neq 0$ $vx \neq 0, \varepsilon = 0.5$	$mx, my, mz \neq 0$ $vx \neq 0, \varepsilon = 1.5$	$mx, my, mz \neq 0$ $vx \neq 0, \varepsilon = 3$	
SR	94.01%	92.83%	91.61%	
ED	27.6 m	33.3 m	36.2 m	

4.6. Different Numbers of UAVs

Table 6 presents the CR, SR, and DR performances in scenarios with different numbers of UAVs J ∈ {2, 4, 8, 10, 12, 20}. The rates are averaged over 300 random realizations (all UAVs having random starting points and destinations). From the table, it can be noted that with increasing numbers of UAVs, the SR decreases due to the higher risk of collision, the ED has an overall upward trend, while the CE is not affected by the number of UAVs.

Table 6. SR, CE, and ED performance with different numbers of UAVs.

	J = 2	J = 4	J = 8	J = 10	J = 12	J = 20
SR	100%	99.1%	95.03%	95.62%	92.10%	90.56%
CE	0.0832 s	0.0721 s	0.0963 s	0.1861 s	0.0910 s	0.216 s
ED	20.3 m	25.2 m	26.8 m	27.6 m	39.1 m	38.2 m

We note that when there are 20 UAVs in an urban scenario of 1 square kilometer, our method still maintains a success rate of more than 90% in terms of conflict, indicating that this method can provide safe guidance for 20 UAVs in that area, which is sufficient to adapt to the current scale of urban UAVs.

In Figure 14, illustrations of collision avoidance in scenarios with different numbers of UAVs J ∈ {2, 8, 20} are presented. From Figure 14, it can be observed that due to the different locations and numbers of UAVs, the agent may determine different avoidance actions when facing collision risks, leading to different trajectories.

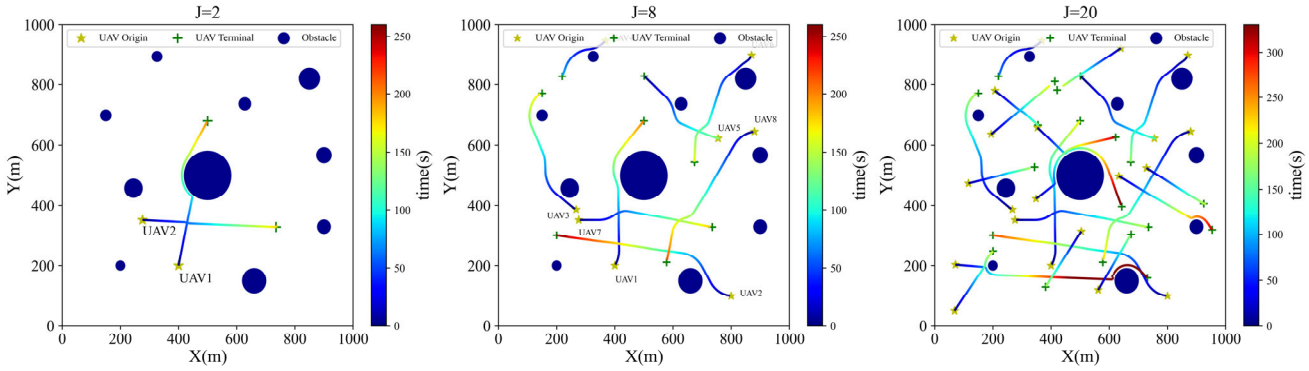


Figure 14. Illustrations of collision avoidance in different scenarios that J ∈ {2, 8, 20}.

4.7. Comparison with Other Algorithms

In the same scenario, two other algorithms are selected for comparison: the double deep Q network (DDQN) and the artificial potential field (APF).

DDQN is a typical value-based DRL algorithm [18,19], while APF utilizes the repulsive force of obstacles and the gravitational force of target to guide the UAV motion, and is widely used in research on collision avoidance [30,31]. DDQN requires that the agent action space be discrete, which is set to $\Delta\varphi \in \{-3°, 0, 3°\}$, $\Delta Z \in \{-1\text{ m}, 0\text{ m}, 1\text{ m}\}$, $\Delta V \in \{-2\text{ m/s}, 0\text{ m/s}, 2\text{ m/s}\}$. The APF action space is consistent with our method. Experiments using DDQN were conducted based on a two−layer resolution framework, and the APF was divided into two categories: APF with a two−layer framework and APF without a two−layer framework. Table 7 presents the performances of the different algorithms.

Table 7. SR, CE, and ED performance of different algorithms and J = 8.

	SR	CE	ED
APF without two−layer framework	89.32%	1.8329 s	41.2 m
APF with two−layer framework	91.65%	0.0821 s	34.3 m
DDQN	93.83%	0.1839 s	56.3 m
Improved DDPG	95.03%	0.0963 s	26.8 m

The improved DDPG algorithm has an absolute advantage in terms of the SR, which is much greater than the other algorithms, at 95.03%. There was not much difference in CE, and only APF was higher than the other algorithms. Our method achieved the minimum ED, which was the largest value in DDQN; because of the constraints of the discrete action space, the DDQN-trained agent can only perform a limited number of action values and the flexibility of the UAV cannot be fully utilized. In addition, the performance of the APF

with a two−layer framework was superior to that of the original APF algorithm, indicating that the two−layer resolution framework can better avoid conflict risks when presented with mUAVs.

It is worth noting that we observed in training sessions that the improved DDPG completed convergence faster than the DDQN, which is consistent with our results in Section 4.2, compared to the original DDPG algorithm, and further indicates that the dynamic adjustment mechanism can reduce the number of training episodes.

5. Conclusions

In this paper, an adaptive method for mUAV collision avoidance in urban air traffic was studied. The main conclusions are as follows:

Firstly, the proposed two−layer resolution framework provides a new concept for realizing mUAV collision avoidance, in which each UAV is endowed with decision-making ability, and the computational complexity is controlled at the polynomial level. Using the improved DDPG algorithm to train the agent allows convergence to be completed faster, which saves training costs to a great extent.

Secondly, the numerical results indicate that the proposed method is able to adapt to various scenarios, e.g., different numbers and positions of UAVs, and interference from random factors. More specifically, the average decision time of the method is 0.0963 s with eight UAVs, the overall resolution success rate is 95.03%, and the extra flight distance is 26.8 m. Our method has better performance when compared to APF, APF with a two−layer framework, and DDQN.

Thirdly, from the perspective of the avoidance process, changing the heading angle is the main way of avoiding collision, the minimum distance from buildings is about 7 m, and the minimum distance from other UAVs is about 4 m, which further proves that the method has a relatively high sensitivity for static obstacles. Our future research focus will be on how to determine the appropriate safety interval and how to reflect this in the resolution process.

Despite the strengths of our proposed approach, there are some drawbacks that require further study. In this paper, distance and velocity vector were calculated for conflict detection, which lacks objective quantification of conflict risk and may have an impact on the subsequent collision avoidance. The quantitative assessment of UAV conflict risk based on multiple factors could guarantee a more accurate determination of conflict targets and resolution strategies, which would be a valuable research direction in the future. Another valuable research direction would be to combine kinematics theory and control theory. Assigning appropriate resolution strategies for UAV collision avoidance at the level of urban air traffic management, while designing UAV controllers from the perspective of control performance, thus ensuring that UAVs can successfully complete avoidance actions by formulating suitable control parameters and suppressing the influence of external disturbances [32,33]. This would promote the engineering application of the method described in this paper.

Author Contributions: Conceptualization, H.Z.; methodology, J.Z. (Jinpeng Zhang) and H.Z.; software, J.Z. (Jinpeng Zhang) and J.Z. (Jinlun Zhou); validation, J.Z. (Jinpeng Zhang), M.H. and G.Z.; formal analysis, J.Z. (Jinpeng Zhang) and H.L.; investigation, M.H. and J.Z. (Jinlun Zhou); resources, J.Z. (Jinpeng Zhang), H.Z. and H.L.; writing—original draft preparation, J.Z. (Jinpeng Zhang); writing—review and editing, J.Z. (Jinpeng Zhang) and M.H.; visualization, J.Z. (Jinpeng Zhang) and M.H. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by National Natural Science Foundation of China (71971114).

Data Availability Statement: Data will be made available on request.

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

D_u, D_o	The collision zone of UAV and building
φ	The heading angle, horizontal speed and latitude
V	The horizontal speed of UAV
Z	The latitude of UAV
φ_g	The relative heading angle of the destination to UAV
d_g	The horizontal distance of the destination to UAV
φ_{us}	The difference in heading angle between two UAV
Z_{us}	The difference in altitude between two UAV
d_{us}	The horizontal distance between two UAV
d_{det}	The UAV detection distance
$\pi(s;\theta)$	Policy network
$\pi(s;\theta^-)$	Target policy network
$\pi_\theta(s_i)$	The learned policy function
i, j	Superscript or subscript, denote the specific UAV
$\Delta\varphi$	Alteration in direction
ΔZ	Alteration in altitude
ΔV	Alteration in horizontal speed
d_{om}	The distance attribute of obstacle in sector m
R_d	Destination intent or collision avoidance reward
R_{uu}	The UAV collision avoidance reward
R_{ex}	The additional reward
S	The state space of agent
A	The action space of agent
P	The collision resolution pool
$Q(s,a;\omega)$	Q network
$Q(s,a;\omega^-)$	Target Q network
$Q_\omega(s,a)$	The learned value function
t	Subscript, denote the specific moment

References

1. Garrow, L.A.; German, B.J.; Leonard, C.E. Urban air mobility: A comprehensive review and comparative analysis with autonomous and electric ground transportation for informing future research. *Transp. Res. Part C Emerg. Technol.* **2021**, *132*, 103377. [CrossRef]
2. Barrado, C.; Boyero, M.; Brucculeri, L.; Ferrara, G.; Hately, A.; Hullah, P.; Martin-Marrero, D.; Pastor, E.; Rushton, A.P.; Volkert, A. U-Space Concept of Operations: A Key Enabler for Opening Airspace to Emerging Low-Altitude Operations. *Aerospace* **2020**, *7*, 24. [CrossRef]
3. 2022 Civil Aviation Development Statistical Bulletin. 2023. Available online: https://file.veryzhun.com/buckets/carnoc/keys/7390295f32633128e6e5cee44fc9fe4e.pdf (accessed on 1 May 2023).
4. Kiran, B.R.; Sobh, I.; Talpaert, V.; Mannion, P.; Sallab, A.A.A.; Yogamani, S.; Pérez, P. Deep Reinforcement Learning for Autonomous Driving: A Survey. *IEEE Trans. Intell. Transp. Syst.* **2022**, *23*, 4909–4926. [CrossRef]
5. Wu, Y. A survey on population-based meta-heuristic algorithms for motion planning of aircraft. *Swarm Evol. Comput.* **2021**, *62*, 100844. [CrossRef]
6. Zeng, D.; Chen, H.; Yu, Y.; Hu, Y.; Deng, Z.; Leng, B.; Xiong, L.; Sun, Z. UGV Parking Planning Based on Swarm Optimization and Improved CBS in High-Density Scenarios for Innovative Urban Mobility. *Drones* **2023**, *7*, 295. [CrossRef]
7. Zhao, P.; Erzberger, H.; Liu, Y. Multiple-Aircraft-Conflict Resolution Under Uncertainties. *J. Guid. Control Dyn.* **2021**, *44*, 2031–2049. [CrossRef]
8. Yun, S.C.; Ganapathy, V.; Chien, T.W. Enhanced D* Lite Algorithm for mobile robot navigation. In Proceedings of the 2010 IEEE Symposium on Industrial Electronics and Applications (ISIEA), Penang, Malaysia, 3–5 October 2010; pp. 545–550.
9. Wu, Y.; Low, K.H.; Pang, B.; Tan, Q. Swarm-Based 4D Path Planning For Drone Operations in Urban Environments. *IEEE Trans. Veh. Technol.* **2021**, *70*, 7464–7479. [CrossRef]
10. Zhang, Q.; Wang, Z.; Zhang, H.; Jiang, C.; Hu, M. SMILO-VTAC Model Based Multi-Aircraft Conflict Resolution Method in Complex Low-Altitude Airspace. *J. Traffic Transp. Eng.* **2019**, *19*, 125–136.
11. Radmanesh, M.; Kumar, M. Flight formation of UAVs in presence of moving obstacles using fast-dynamic mixed integer linear programming. *Aerosp. Sci. Technol.* **2016**, *50*, 149–160. [CrossRef]

12. Waen, J.D.; Dinh, H.T.; Torres, M.H.C.; Holvoet, T. Scalable multirotor UAV trajectory planning using mixed integer linear programming. In Proceedings of the 2017 European Conference on Mobile Robots (ECMR), Paris, France, 6–8 September 2017; pp. 1–6.
13. Alonso-Ayuso, A.; Escudero, L.F.; Martín-Campo, F.J. An exact multi-objective mixed integer nonlinear optimization approach for aircraft conflict resolution. *TOP* **2016**, *24*, 381–408. [CrossRef]
14. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [CrossRef] [PubMed]
15. Silver, D.; Huang, A.; Maddison, C.J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. Mastering the game of Go with deep neural networks and tree search. *Nature* **2016**, *529*, 484–489. [CrossRef]
16. Pham, H.X.; La, H.M.; Feil-Seifer, D.; Nguyen, L.V. Autonomous uav navigation using reinforcement learning. *arXiv* **2018**, arXiv:1801.05086.
17. Liu, X.; Liu, Y.; Chen, Y. Reinforcement Learning in Multiple-UAV Networks: Deployment and Movement Design. *IEEE Trans. Veh. Technol.* **2019**, *68*, 8036–8049. [CrossRef]
18. Singla, A.; Padakandla, S.; Bhatnagar, S. Memory-Based Deep Reinforcement Learning for Obstacle Avoidance in UAV with Limited Environment Knowledge. *IEEE Trans. Intell. Transp. Syst.* **2021**, *22*, 107–118. [CrossRef]
19. Zhai, P.; Zhang, Y.; Shaobo, W. Intelligent Ship Collision Avoidance Algorithm Based on DDQN with Prioritized Experience Replay under COLREGs. *J. Mar. Sci. Eng.* **2022**, *10*, 585. [CrossRef]
20. Li, C.; Gu, W.; Zheng, Y.; Huang, L.; Zhang, X. An ETA-Based Tactical Conflict Resolution Method for Air Logistics Transportation. *Drones* **2023**, *7*, 334. [CrossRef]
21. Lillicrap, T.P.; Hunt, J.J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; Wierstra, D. Continuous control with deep reinforcement learning. *arXiv* **2015**, arXiv:1509.02971.
22. Ribeiro, M.; Ellerbroek, J.; Hoekstra, J. Improvement of conflict detection and resolution at high densities through reinforcement learning. In Proceedings of the ICRAT2020: International Conference on Research in Air Transportation 2020, Tampa, FL, USA, 23–26 June 2020.
23. Rubí, B.; Morcego, B.; Pérez, R. Deep reinforcement learning for quadrotor path following with adaptive velocity. *Auton. Robot.* **2021**, *45*, 119–134. [CrossRef]
24. Zhang, Y.; Zhang, Y.; Yu, Z. Path Following Control for UAV Using Deep Reinforcement Learning Approach. *Guid. Navig. Control* **2021**, *1*, 18. [CrossRef]
25. Wen, H.; Li, H.; Wang, Z.; Hou, X.; He, K. Application of DDPG-based Collision Avoidance Algorithm in Air Traffic Control. In Proceedings of the 2019 12th International Symposium on Computational Intelligence and Design (ISCID), Hangzhou, China, 14–15 December 2019; pp. 130–133.
26. Hu, J.; Yang, X.; Wang, W.; Wei, P.; Ying, L.; Liu, Y. Obstacle Avoidance for UAS in Continuous Action Space Using Deep Reinforcement Learning. *IEEE Access* **2022**, *10*, 90623–90634. [CrossRef]
27. Zhang, H.; Zhang, J.; Zhong, G.; Liu, H.; Liu, W. Multivariate Combined Collision Detection for Multi-Unmanned Aircraft Systems. *IEEE Access* **2022**, *10*, 103827–103839. [CrossRef]
28. Qiu, C.; Hu, Y.; Chen, Y.; Zeng, B. Deep Deterministic Policy Gradient (DDPG)-Based Energy Harvesting Wireless Communications. *IEEE Internet Things J.* **2019**, *6*, 8577–8588. [CrossRef]
29. Bagdi, Z.; Csámer, L.; Bakó, G. The green light for air transport: Sustainable aviation at present. *Cogn. Sustain.* **2023**, *2*. [CrossRef]
30. Guo, Y.; Liu, X.; Jiang, W.; Zhang, W. Collision-Free 4D Dynamic Path Planning for Multiple UAVs Based on Dynamic Priority RRT* and Artificial Potential Field. *Drones* **2023**, *7*, 180. [CrossRef]
31. Sun, J.; Tang, J.; Lao, S. Collision Avoidance for Cooperative UAVs with Optimized Artificial Potential Field Algorithm. *IEEE Access* **2017**, *5*, 18382–18390. [CrossRef]
32. Song, J.; Hu, Y.; Su, J.; Zhao, M.; Ai, S. Fractional-Order Linear Active Disturbance Rejection Control Design and Optimization Based Improved Sparrow Search Algorithm for Quadrotor UAV with System Uncertainties and External Disturbance. *Drones* **2022**, *6*, 229. [CrossRef]
33. Bauer, P.; Ritzinger, G.; Soumelidis, A.; Bokor, J. LQ servo control design with Kalman filter for a quadrotor UAV. *Period. Polytech. Transp. Eng.* **2008**, *36*, 9–14. [CrossRef]